# DiffusEdit: Technical Design

**Tatva Agarwal***, **Vedant Kulkarni***, **Byasani Vedavyas***, **Venya Velmurugan***

IIIT Hyderabad

*Equal contribution; authors listed alphabetically.

Reference articles go out of date as events develop, and updating them by hand is slow. Asking a language model to write a fresh article automates the update, but it rewrites text that was already correct and leaves no record of what changed. DiffusEdit instead edits the article in place. Small trained layers, reading the hidden states of a frozen masked diffusion language model, decide which sentences to keep, edit, or delete, and which tokens inside them are stale. The frozen model rewrites only the stale positions, and the system repeats the process on sentences that the rewrites themselves made wrong. Training labels come from pairs of Wikipedia snapshots, and evaluation measures update quality, faithfulness to the evidence, closeness to the original article, and coverage of the forced secondary edits. This document details the proposed technical design of the project. Every numeric value in this document is a starting point for validation rather than a final choice.

**Date:** August 28, 2026

**Correspondence:** vedant.kulkarni@students.iiit.ac.in

## 1 Introduction

Reference text goes out of date. A Wikipedia article about a factory fire may say that three people died and that the small death toll limited the political fallout. A week later, news reports revise the toll to forty-two. The first sentence is now wrong in an obvious way, because the evidence contradicts it directly. The second sentence is wrong in a quieter way. No news report mentions political fallout at all, yet the sentence rests on a fact that no longer holds. An editor who fixes the death toll and stops has left the article internally inconsistent. We call the second kind of required change a *cascading edit*, a sentence that must change only because another sentence changed.

Keeping such articles current by hand is slow. The editor must read the incoming evidence, find every affected sentence, rewrite each one without disturbing the rest, and then re-read the article for sentences that the rewrites broke. DiffusEdit automates this process. Given the outdated article and the new evidence, it outputs an updated article, and it aims to change as little text as possible along the way.

The standard automated alternative is to hand the article and the evidence to a language model and ask it to write a fresh article. This works, but it has two costs. First, text that was already correct gets rewritten, and rewriting is risky, because models sometimes introduce errors into sentences that needed no change. Second, the output carries no record of what changed, so a human reviewer must compare the old and new articles line by line. DiffusEdit avoids both costs by editing in place, so correct text survives verbatim and the changed positions are known exactly because the system chose them.

The rest of this document builds the system up from its parts. Section 2 introduces the machine learning background: tokens, language models, masked diffusion models, hidden states, and classifiers. Readers who know these topics can skim it and pick up the notation. Section 3 states the task precisely, and Section 4 places the design among existing systems. Section 5 walks through the system once at a high level. Sections 6 and 7 give the full technical detail of the scoring components and the edit loop. Section 8 explains where the training data comes from and how we label it. Section 9 explains how we will measure success, starting from what precision and recall mean. Section 10 lists possible failures and how we would deal with them.

# 2 Background

## 2.1 Tokens and Language Models

Neural networks that process text first split it into *tokens*, short pieces, usually a word or part of a word, drawn from a fixed list called the *vocabulary*. A vocabulary $V$ typically holds on the order of a hundred thousand tokens. The sentence *Three people died in the fire* might become the seven tokens (Three, people, died, in, the, fire, .). From here on, a piece of text and its token sequence are interchangeable.

A *language model* is a neural network that assigns probabilities to tokens. Given some visible text, it outputs, for a position of interest, a number between zero and one for every vocabulary entry, and the numbers sum to one across the vocabulary. A high number on a token means the model considers that token likely at that position. Training such a model means showing it a vast quantity of real text and adjusting its internal weights so that the probability it assigns to the text it sees goes up. The result is a model that has absorbed the patterns of its training text: grammar, facts, style, and the statistics of which words follow which.

Most familiar language models are *autoregressive*, meaning they generate text left to right, choosing one token at a time, each choice conditioned only on the tokens to its left. This design generates fluent text, but it fits editing poorly. If we want to rewrite three tokens in the middle of a paragraph, an autoregressive model gives us no direct way to condition on the text that comes *after* the gap, because it never learned to look rightward.

## 2.2 Masked Diffusion Language Models

A *masked diffusion language model* learns a different skill, filling in blanks. During training, some fraction of the tokens in a text are replaced by a placeholder symbol, written m and called the *mask*. The model sees the partially masked text and must predict the original token at every masked position, using the visible tokens on *both* sides. Corrupting text with masks and learning to undo the corruption is the idea behind the name. Masking plays the role of noise, and the model learns to reverse it, in loose analogy with diffusion models for images. LLaDA (Nie et al., 2025) is the model we build on. Dream (Ye et al., 2025) is a drop-in alternative.

Formally, let $x_0 = (x_0^1, \ldots, x_0^L)$ be a text of $L$ tokens. The corruption process picks a noise level $t$ between zero and one, then replaces each token independently by m with probability $t$:

$$q(x_t \mid x_0) \;=\; \prod_{i=1}^{L} \Big[ (1-t)\,\mathbf{1}\big[x_t^i = x_0^i\big] + t\,\mathbf{1}\big[x_t^i = \mathrm{m}\big] \Big]. \tag{1}$$

Here $\mathbf{1}[\cdot]$ equals one when the bracketed statement holds and zero otherwise, so each factor says that the token survives with probability $1-t$ and becomes m with probability $t$. The model $p_\theta$, with weights $\theta$, learns to predict the originals at the masked spots by minimizing

$$\mathcal{L}(\theta) \;=\; -\,\mathbb{E}_{t,\,x_0,\,x_t} \left[ \frac{1}{t} \sum_{i=1}^{L} \mathbf{1}\big[x_t^i = \mathrm{m}\big]\; \log p_\theta\big(x_0^i \mid x_t\big) \right]. \tag{2}$$

Reading Equation (2) from the inside out works as follows. For every masked position $i$, take the probability the model assigns to the true original token, and take its logarithm, so that confident correct predictions cost little and confident wrong ones cost much. Sum over the masked positions and scale by $1/t$, which balances the contributions of light and heavy corruption. The symbol $\mathbb{E}_{t,\,x_0,\,x_t}$ is an *expectation*, an average over the random quantities named in its subscript: the noise level $t$, drawn uniformly between zero and one; the text $x_0$, drawn from the training corpus; and the corruption $x_t$, drawn from Equation (1). Finally, the leading minus sign turns the goal of making probabilities large into the goal of making the loss small, which is the direction gradient descent works in. Training pushes this loss down, which pushes the model's predictions toward the original text.

To *generate* text, one starts from a sequence that is all m and repeatedly asks the model to predict every masked position, committing the predictions it is most confident about and leaving the rest masked, over $T$ rounds until nothing is masked. To *infill*, one runs the same procedure on a sequence where only chosen

positions are masked. Infilling is the operation DiffusEdit relies on, since the model rewrites exactly the positions we mask while conditioning on everything around them.

Every mask commits to exactly one token, a structural property that matters throughout this document. Masking five positions yields a replacement of exactly five tokens. A correct rewrite is often shorter or longer than the text it replaces, so Section 7.2 works around this constraint.

Masked diffusion is not the only architecture that fits editing. An encoder-decoder model such as T5 also reads context on both sides and emits variable-length output. Two properties motivate the choice. The same frozen model supplies both the hidden states that every scoring head reads and the infilling engine, so the system contains no separately trained generator. And infilling changes only the masked positions, so untouched text survives exactly, where an encoder-decoder that re-emits a sentence may vary it. Whether these properties hold up at article scale is part of what the project tests.

### 2.3 Hidden States

A neural language model processes text through a stack of layers. At each layer, every token position carries a vector of numbers, and each layer transforms these vectors using the surrounding context. The vector at position $i$, taken from some layer, is the *hidden state* $h_i \in \mathbb{R}^d$. For LLaDA-8B the dimension $d$ is 4096. A hidden state is a numeric summary of that token *in context*. Two occurrences of the word *bank* get different hidden states in *river bank* and *bank account*, because the surrounding words differ.

Hidden states are the interface between the frozen model and everything DiffusEdit trains. We never update the weights of the language model itself. We say it stays *frozen*. Instead we run one forward pass, keep the hidden states, and train small components that read them. This is standard practice under the name *probing*, and it is cheap, because the expensive model runs once per input and the trainable parts are tiny.

Unless stated otherwise, hidden states come from one chosen layer $\ell^*$. The default is the final layer. We will also try the layer at three quarters of the depth, because middle-to-late layers often carry more transferable features than the last one.

### 2.4 Classifiers

A *classifier* is a function that takes a vector and returns a probability for each of a fixed set of labels. The simplest useful classifier is a single linear layer. For two labels, it computes a weighted sum of the input vector's entries and squashes the result into $(0, 1)$ with the logistic function $\sigma(z) = 1/(1 + e^{-z})$. The output reads as the probability of the positive label. For $k$ labels, it computes $k$ weighted sums and normalizes them into $k$ probabilities with the *softmax* function, which exponentiates each sum and divides by the total.

Training a classifier requires *labels*, the answer we want for each training input. A *loss function* measures the mismatch between predicted probabilities and true labels. Ours is *cross-entropy*, the negative logarithm of the probability the classifier put on the correct label. Gradient descent then adjusts the classifier's weights step by step to shrink the average loss. All of this is cheap for the classifiers in this document, because each has at most a few million weights, against eight billion in the frozen language model.

Data is always split three ways. The *training set* fits the weights. The *validation set* chooses the settings that training does not set, such as thresholds, and provides an early warning when a model memorizes its training data instead of learning the pattern, a failure called *overfitting*. The *test set* is touched once, at the end, to report the final numbers. A number tuned on test data no longer measures generalization, so every threshold in this document comes from validation data.

## 3 Task Definition

An article is a sequence of sentences $A = (s_1, \ldots, s_n)$. Flattening the sentences gives the token sequence $x = (x_1, \ldots, x_L)$. New evidence arrives as a set of documents $E = (e_1, \ldots, e_m)$. In our data these are text snippets, sometimes prose and sometimes a table rendered as text. Call the article as it should read after the update $A^*$. The system never sees $A^*$ and must produce its own updated article $\hat{A}$ from $A$ and $E$ alone.

The output must balance three requirements. *Faithfulness* requires every claim in $\hat{A}$ to agree with the evidence and with the untouched parts of the original article. *Coverage* requires every evidence fact that bears on the article to appear in $\hat{A}$. *Minimality* requires $\hat{A}$ to stay as close to $A$ as possible. Regenerating from scratch can satisfy the first two while failing minimality badly, and deleting every contested claim would satisfy faithfulness while failing coverage. We state the goal as a constrained minimization:

$$\hat{A} \;=\; \arg\min_{A'} \; d(A', A) \quad \text{subject to} \quad \text{faithful}(A', E) \text{ and } \text{covered}(A', E), \tag{3}$$

where $d$ counts token-level edits between two texts, faithful is the predicate that every claim in $A'$ agrees with $E$, and covered is the predicate that every evidence fact bearing on the article appears in $A'$. Neither predicate is computable. They are idealized targets, and the metrics of Section 9.2 serve as their measurable stand-ins, AlignScore and the fact verifier for faithful, UpdateROUGE and entity recall for covered. One consequence for the DROP label follows. Deleting a sentence is the correct repair only when the evidence retracts its fact, and not when the evidence merely contradicts the current phrasing, because the corrected fact still belongs in the article under covered. The approximation is greedy. It finds the smallest set of positions that appear to violate the constraints, rewrites only those, and repeats until nothing appears to violate them.

Write $s_j \rightsquigarrow s_k$ when changing $s_j$ forces a change in $s_k$ even though the evidence says nothing about $s_k$ directly. The fire example from Section 1 shows the pattern, with $s_j$ the death-toll sentence and $s_k$ the fallout sentence. A single editing pass repairs only sentences the evidence contradicts. Sentences reachable through $\rightsquigarrow$ stay stale, and that is the published limitation of Detect, Remask, Repair (Zou et al., 2026), the single-pass system that DiffusEdit extends from short summaries to multi-paragraph articles.

# 4 Related Work

*Updating text against new evidence.* The oldest work in this area corrects one sentence or one claim at a time. Shah et al. (2020) take a sentence that a new fact contradicts, mask the tokens that clash with the fact, and let a model fill the gap, which is the same mask-and-fill idea DiffusEdit uses inside a single sentence. Thorne and Vlachos (2021) do the same for short factual claims checked against retrieved evidence. Chen et al. (2022) reach the corrected claim step by step, making one small edit at a time until the claim agrees with the evidence. Iso et al. (2020) correct text against a table of facts instead of prose evidence. A related line fixes the output of language models rather than human-written articles. These systems check each claim the model generated against retrieved evidence and revise the claims that fail the check (Gao et al., 2023; Krishna et al., 2024). PEER (Schick et al., 2022) learns from Wikipedia's edit history to follow written instructions such as *fix the outdated numbers*, planning and writing the edit itself. Closest to us, FRUIT (Logan IV et al., 2022) defines the exact task this project works on, updating a whole article from evidence, and its EDIT5 model writes the updated article left to right while copying unchanged sentences through special tokens. Across all of these, the editing unit is either a single sentence or the whole regenerated text. None of them marks stale tokens inside the article, and none goes back over its own edits to catch cascades.

*Edit propagation.* Two recent works address the cascade problem directly. EditPropBench (Kruthof, 2026) is a benchmark, not a system. It plants a factual change in a synthetic scientific paper, records every downstream sentence that must change with it, and measures how many of those a model finds. The strongest prompted model tested misses about 30% of the required follow-on changes when the dependence is implicit, so the problem is open. HiCaM (Shi et al., 2025) is a system. It has a large prompted model build two maps of the document, a graph of which entities affect which, and a tree of summaries at several levels of detail, and it then uses the maps to push one change through to the connected content. Both approaches call a large model many times per document. DiffusEdit instead makes the propagation decision with small trained heads reading cached hidden states (Section 7.3), so the expensive model stays out of that decision.

*Editing with diffusion models.* A growing line of work lets a diffusion model reconsider tokens it has already committed during generation. ReMDM (Wang et al., 2025) derives rules for putting masks back during sampling. LLaDA2.1 (Bie et al., 2026) finds low-confidence tokens and overwrites them with better guesses. Yao (2026) argue that overwriting is the wrong action, because a wrong committed token pulls nearby

predictions in its direction while a mask pulls in no direction, so the better action is to re-mask the token and let the model predict it again from cleaner context. Two of their tools appear in DiffusEdit. Their cap on how often one position may be re-masked is our loop budget (Section 7.4), and their trigger, a token whose confidence falls as decoding proceeds, is the single-run version of our staleness delta across rounds (Section 7.3). Another branch works on the one-token-per-mask constraint from Section 2.2. DreamOn (Wu et al., 2026) and DDOT (Zhang et al., 2025) train models whose masked region can grow or shrink while infilling, which would remove the need for our candidate lengths but requires training the diffusion model, and our design keeps it frozen. All of these fix mistakes the model makes while generating fresh text. Detect, Remask, Repair (Zou et al., 2026) is the one prior system that applies a diffusion model to existing text and external evidence, and it is the system DiffusEdit extends.

*Knowledge editing.* A different research line updates the model instead of the text. There, an outdated fact lives inside a model's weights, and editing means changing those weights so the model recalls the new fact (Meng et al., 2022). In DiffusEdit, the article carries the outdated fact, the article is what gets corrected, and every network in the system keeps its weights fixed.

# 5 System Overview

DiffusEdit runs in rounds. Each round has four stages, and every stage reads the hidden states of one frozen-model forward pass over the evidence and the current article.

1. *Classify.* A sentence classifier labels every sentence KEEP (leave it alone), EDIT (some of its tokens must change), or DROP (delete the whole sentence). Deletion needs its own label because infilling replaces tokens but cannot remove them.
2. *Mask.* Inside each EDIT sentence, a token-level detector scores every token for staleness, and tokens scoring above a threshold become m.
3. *Infill.* The frozen diffusion model fills the masked spans, conditioned on the evidence and on all the unmasked text. Each span is tried at several replacement lengths, and the best-scoring completion wins.
4. *Cascade.* A cascade detector looks at what just changed and flags sentences elsewhere in the article that the changes may have broken. Flagged sentences enter the next round as EDIT sentences.

The loop ends when a round flags nothing new, or when budget caps stop it (Section 7.4).

On the fire example, round one classifies the death-toll sentence EDIT because the evidence contradicts it, masks the token *three*, and infills *forty-two*. The classifier leaves the fallout sentence as KEEP in round one, since no evidence mentions it. After the repair, the cascade detector flags the fallout sentence, because it shares content with a sentence that just changed. Round two masks the now-unsupported phrase about a small death toll limiting the fallout and rewrites it. Round three flags nothing, and the loop ends.

Three small trained components make the decisions: the sentence classifier, the token staleness detector, and one variant of the cascade detector. Section 6 specifies them. The loop logic and the infilling machinery around the frozen model follow in Section 7.

# 6 Scoring Heads

We call each small trained component a *head*, a small network that reads the frozen model's hidden states. This section gives each head's input, output, architecture, and training objective. Training data for all of them comes from Section 8.

## 6.1 Sentence Pooling

Heads that judge whole sentences need one vector per sentence, but the model produces one per token. We average the token vectors. For sentence $s_j$ covering token positions $\mathcal{I}_j$,

$$u_j \;=\; \frac{1}{|\mathcal{I}_j|} \sum_{i \in \mathcal{I}_j} h_i. \tag{4}$$

Averaging vectors keeps what the token vectors agree on and washes out position-specific detail. This is *mean pooling*. If validation results are poor, the fallback is *attention pooling*, where a learned query vector decides how much each token contributes instead of weighting all tokens equally.

## 6.2 Sentence Classifier

The sentence classifier is a linear layer mapping the pooled vector $u_j$ from Equation (4) to three probabilities:

$$p_j \;=\; \mathrm{softmax}(W_c\, u_j + b_c)\,, \qquad W_c \in \mathbb{R}^{3 \times d}, \tag{5}$$

one probability each for KEEP, EDIT, and DROP. For KEEP and EDIT, the predicted label is the arg-max. DROP is held to a stricter rule, because later rounds cannot undo a deletion. The system cannot insert sentences, so a wrong deletion is permanent. A sentence is deleted only when its DROP probability clears a separate confidence gate $\tau_d$, tuned on validation, and a sentence that favors DROP without clearing the gate is treated as EDIT instead. Revision history also deletes sentences for restructuring and style, so the derived DROP labels are broader than the retraction rule of Section 3. The gate absorbs part of that mismatch, and the label audit (Section 8.4) records how often a derived DROP is a genuine retraction.

Most sentences in a real update are KEEP, since FRUIT articles average three to four updated sentences out of roughly ten (Section 8.2). A classifier trained on skewed labels learns a shortcut. It predicts the common class for every input, which scores well on the skewed label distribution while carrying no information. The standard fix is to make rare-class mistakes cost more. With $N$ training sentences of which $N_c$ carry class $c$, the weighted cross-entropy loss is

$$\mathcal{L}_{\mathrm{cls}} \;=\; -\sum_j \omega_{y_j}\, \log p_j[y_j], \qquad \omega_c \;=\; \frac{N}{3\, N_c}, \tag{6}$$

where $y_j$ is the true label of sentence $j$ and $p_j[y_j]$ is the probability the classifier gave it. The weight $\omega_c$ grows as class $c$ gets rarer, so the classifier cannot reach a low loss by ignoring EDIT and DROP.

## 6.3 Token Staleness Scoring

Inside each EDIT sentence we need to know *which* tokens to mask. The token scorer, following the [MASK]-DISC detector of Zou et al. (2026), is a linear layer with a sigmoid output on each token's hidden state:

$$\pi_i \;=\; \sigma\big(w_d^\top h_i + b_d\big)\,, \tag{7}$$

read as the probability that token $i$ is stale. It trains with binary cross-entropy, the two-class version of the loss from Section 2.4, against stale-or-not labels from two sources.

1. *Derived labels.* Section 8.4 aligns each outdated article with its corrected version and diffs aligned sentence pairs token by token. Tokens that the real revision replaced or removed are labeled stale.
2. *Synthetic corruptions.* Real labeled data is finite and noisy, so we manufacture more, following Zou et al. (2026). Take a correct article. Sample a corruption fraction $t$ uniformly from $[0.3, 0.7]$, mask that fraction of its tokens, and let the frozen model refill the masked spots while conditioning on the *outdated* evidence. The refilled tokens are fluent but often unsupported, exactly the failure mode the detector must catch at inference time. Label a refilled token stale when it differs from the original. This yields unlimited training pairs whose error distribution matches the errors our own repair stage produces.

The mixing ratio between the two sources is an experiment axis (Section 9.4). The default is an even mix.

### 6.4 Consequence Prediction

The third head supports cascade detection. It answers a yes-or-no question about an *ordered pair* of sentences. Given that the first sentence just changed, must the second one change too? Its input combines the pooled vectors $u_j$ (the changed sentence) and $u_k$ (the candidate), both from Equation (4):

$$g_{jk} \;=\; \sigma\Big(\mathrm{MLP}_\psi\big([\,u_j;\; u_k;\; u_j \odot u_k;\; |u_j - u_k|\,]\big)\Big). \tag{8}$$

The four blocks inside the brackets are the two vectors themselves, their elementwise product, and their elementwise absolute difference. The product and difference terms give the network explicit similarity signals it would otherwise have to learn on its own. This pair-feature construction is standard for sentence-pair tasks. The MLP (multi-layer perceptron, a stack of linear layers with nonlinearities between them) has one hidden layer of width 512. The subscript $\psi$ names the MLP's trainable weights, in the same way $\theta$ names the language model's. The whole head has about $4d \times 512$ weights and trains in minutes on cached hidden states.

Positive training pairs come from the mining procedure of Section 8.5, which pairs a directly contradicted edit with a consequential edit from the same article. Training encodes each pair the way inference will see it. The direct edit enters in its corrected version, because at inference the changed sentence has already been rewritten, and the candidate enters in its outdated version, because at inference the candidate has not been repaired yet. Negative pairs reuse the same first sentence with KEEP sentences of the same article, three negatives per positive, so the head learns to separate consequence from mere co-occurrence in one article.

## 7 Edit Loop

This section specifies one round end to end, then the loop control, and finally the cost model.

### 7.1 Masking Policy

Round $r$ opens with a fresh forward pass over the evidence and the current article, because the previous round changed the text and the old hidden states no longer match the article. Call the resulting token scores $\pi_i^{(r)}$, computed as in Equation (7).

For each EDIT sentence $s_j$, the masked set is every token scoring at or above a threshold:

$$\mathcal{M}_j \;=\; \big\{\, i \in \mathcal{I}_j : \pi_i^{(r)} \geq \tau \,\big\}. \tag{9}$$

Two rules limit the masked set. If $\mathcal{M}_j$ comes out empty even though the classifier wanted the sentence edited, we mask the top-scoring fifth of its tokens, trusting the classifier over the token scores. And $\mathcal{M}_j$ never exceeds half the sentence, because a sentence needing more than that is closer to a full rewrite than a repair.

Masked positions then merge into *spans*, maximal runs of masked tokens, where a run also absorbs a single unmasked token sandwiched between masked ones. Absorbing one-token gaps turns fragmented scores like *three* m *people* m *died* into one contiguous rewrite region rather than two separate holes. The two limit rules and the gap absorption are heuristics. The system counts how often each one fires, and the counts go into the report.

### 7.2 Variable-Length Infilling

A span of $\ell$ masks yields exactly $\ell$ tokens (Section 2.2), and the correct replacement is often shorter or longer. The workaround is to infill the span several times at different lengths and keep the best result. The candidate set for a span of length $\ell$ is

$$\Lambda(\ell) \;=\; \{\,0,\; \lceil \ell/2 \rceil,\; \ell,\; \ell + 2,\; 2\ell\,\}, \tag{10}$$

after removing duplicates. Length zero means the span disappears, which is how the system deletes a phrase without deleting its sentence. Each candidate length runs the infilling sampler for $T = 8$ steps.

Choosing among candidates needs a score, and the score must reward two things at once, fluency in context and agreement with the evidence. For fluency we use the masked pseudo-log-likelihood

$$\mathrm{PLL}(s) \;=\; \frac{1}{|s|}\sum_{i\in s}\log p_\theta\big(s_i \mid s_{\setminus i},\, X_{\mathrm{ctx}}\big)\,. \tag{11}$$

For each token of the candidate sentence in turn, this masks exactly that one position and asks how probable the frozen model considers the true token there, given everything else. Averaging the log-probabilities gives a per-token fluency measure. Computing it costs one masked probe per token, and the probes batch together. Every candidate is scored as a whole sentence, so the deletion candidate competes on equal footing, and a sentence that reads better without the span gets a higher score.

For agreement with the evidence we use a *fact verifier*, a network trained to read a piece of evidence and a claim and to output the probability that the evidence supports the claim. We will use a verifier trained on VitaminC (Schuster et al., 2021), a dataset of evidence-claim pairs built from Wikipedia revisions. The verifier reads one snippet and one sentence at a time. With several snippets in the input, we score the sentence against each snippet separately and keep the highest score, $\mathrm{Ent}(s, E) = \max_{e\in E}\mathrm{Ent}(s, e)$, so one supporting snippet is enough and distractor snippets do not dilute the result.

The final candidate score blends both terms,

$$R(c) \;=\; \lambda\,\mathrm{PLL}(s^c) + (1-\lambda)\,\mathrm{Ent}(s^c, E), \tag{12}$$

where $s^c$ is the sentence after substituting candidate $c$. The blend weight $\lambda$ comes from a validation grid over $\{0.3, 0.5, 0.7\}$.

The deletion candidate needs one extra rule. Removing a contested span also removes the claim the verifier would have checked, so the deletion candidate scores high whenever the remaining sentence is fluent. Length zero therefore wins only when its score beats the best rewrite by a margin tuned on validation. We also record how often length zero wins and how often the gold revision deletes. A large gap between those two rates signals the bias.

Detect, Remask, Repair makes this choice differently. Instead of reranking finished candidates, it guides the sampler during decoding toward text that an external faithfulness scorer rewards, a technique called reward steering, and it also offers a repair model fine-tuned to fix a draft in one step. We choose reranking because it leaves the frozen sampler untouched and keeps the support check swappable. Steering remains the fallback if reranking proves weak.

## 7.3 Cascade Detectors

After infilling, the round checks which other sentences the repairs may have broken. Let $\Delta_r$ be the set of sentences repaired or deleted in round $r$. Several parts of the design also need a *mention extractor* $M$, which works as follows. Run a standard named-entity recognizer (we will use the spaCy library) over the text, keep the names of people, organizations, and places that it finds along with all numbers and dates, lowercase and trim each one, and return the resulting set of strings. On the fire example, $M$ of the death-toll sentence contains the number *three*, and $M$ of the evidence contains *forty-two*.

We will compare three detectors, in increasing order of sophistication. Each flags sentences outside $\Delta_r$.

(a) *Mention overlap.* Flag $s_k$ when $M(s_k)$ shares at least one string with the mentions of the sentences repaired this round. The idea is that dependence usually travels through shared entities and figures. It needs no training and serves as the baseline. We expect high recall and low precision from it.

(b) *Staleness delta.* After the round's repairs, the article is re-encoded (line 7 of Algorithm 1). We call the resulting hidden states *fresh*, in contrast to the states computed before the repairs. Re-score every token with Equation (7) under the fresh states, and flag $s_k$ when one of its tokens now scores $\pi_i \geq \tau_2$ and sits in the top tenth of the article's token scores, where it was not before the repairs. The idea is that a token which became *more* suspicious after a repair is one the repair undermined. We compare ranks rather than raw score differences, because sigmoid scores from two different encodings can shift a

little for reasons unrelated to the text, while ranks ignore any shift that moves all scores together. The same drop-in-confidence trigger appears within a single decoding run in the work of Yao (2026). Here it operates across rounds.

(c) *Consequence prediction.* Flag $s_k$ when the score $g_{jk}$ of Equation (8) passes a threshold $\tau_c$ for any repaired sentence $s_j$. This is the only detector that had to be trained, and the only one that can catch dependence expressed without shared surface mentions, such as a paraphrase of the changed fact.

Flagged sentences enter round $r+1$ as EDIT sentences. If the classifier, under the fresh hidden states, favors DROP for a flagged sentence and clears the confidence gate of Section 6.2, the sentence is deleted instead of re-masked.

### 7.4 Loop Control

Algorithm 1, at the end of this section, assembles the stages. Two budgets bound the loop. The round count is capped at $R = 3$. And each token position may be re-masked at most $C_{\max} = 3$ times across the whole run, the per-position budget of Yao (2026), which blocks the failure mode where the system masks a span, fills it badly, flags it again, and oscillates forever. Both caps are hard limits, so the loop always ends.

### 7.5 Cost Model

The cost unit is one *forward pass*, meaning one push of one input through the frozen model. Everything else, the heads included, is negligible next to it.

Suppose a round touches $k$ spans. It spends one forward pass on scoring, $|\Lambda| \cdot T$ sampler steps per span (Equation (10)), and $|\Lambda| \cdot \bar{\ell}_s$ PLL probes per span (Equation (11)), where $\bar{\ell}_s$ is the sentence length in tokens. With the defaults ($|\Lambda| = 5$, $T = 8$, $k = 3$, $\bar{\ell}_s = 25$), that is $1 + 3 \times 40 = 121$ sampler-style passes plus 375 probes, all at sequence length near 700. Probes and candidates are independent, so they run in *batches*, several inputs through the model at once.

**Algorithm 1** DiffusEdit inference. Thresholds and caps per Sections 7.1, 7.3 and 7.4. The length set $\Lambda$ comes from Equation (10) and the candidate score $R(c)$ from Equation (12).

$H \leftarrow f_\theta(E \oplus A)$; classify sentences; delete gated DROP sentences; $\mathcal{E} \leftarrow$ EDIT set
**for** $r = 1, \ldots, R$ **do**
  **for** $s_j \in \mathcal{E}$ **do**
    mask spans in $s_j$ per policy, skipping tokens with exhausted budgets $c_i = C_{\max}$
    infill each span at every length in $\Lambda(\ell)$; keep the candidate maximizing $R(c)$
  **end for**
  $H \leftarrow f_\theta(E \oplus A^{(r)})$
  $\mathcal{E} \leftarrow$ sentences flagged by the active cascade detector
  **if** $\mathcal{E} = \emptyset$ or no maskable token remains **then break**
  **end if**
**end for**
**return** current article

## 8 Data

### 8.1 Distant Supervision

Supervised learning needs examples with answers. For this task an example is a triple: outdated article, evidence, corrected article. Nobody hand-writes such triples at scale, but Wikipedia's revision history contains millions of them. Whenever editors updated an article between two dates, the earlier version is the outdated article, the later version is the corrected one, and text added elsewhere on Wikipedia in the meantime can serve as evidence.

**Table 1** FRUIT-WIKI statistics (Logan IV et al., 2022). Evidence counts are snippets, not documents. Blank cells are unreported.

| Split | Years | Articles | Edits | Evidence |
|---|---|---|---|---|
| Train (silver) | 2019–2020 | 114K | 407K | 720K |
| Test (silver) | 2020–2021 | 54K | 182K | |
| Test (gold) | 2020–2021 | 914 | 3.0K | |

Labels obtained this way are called *distantly supervised*, meaning an automatic procedure inferred them and no human checked them. Distant supervision trades label quality for volume. Some fraction of the labels are wrong. The convention is to call the automatic labels *silver* and human-verified labels *gold*. The rule we follow everywhere is to train on silver, tune on silver, and report results on gold only.

## 8.2 FRUIT-WIKI

FRUIT-WIKI (Logan IV et al., 2022) is exactly such a dataset, built by diffing the introductory sections of English Wikipedia between yearly snapshots and attaching evidence snippets found elsewhere in the encyclopedia. Table 1 lists the numbers we plan around. Two of them drive design decisions. First, the gold set has only 914 articles, so gold is strictly for final evaluation and would be far too small for training. Second, the FRUIT authors estimate that only about a third of silver edits are supported by their attached evidence. The remaining edits are stylistic rewrites or changes whose real source lies elsewhere. Our derived labels inherit this noise, which is one reason Section 6.3 keeps a synthetic data stream alongside the derived one.

An average article carries three to four updated sentences and about seven evidence snippets, and many snippets are irrelevant distractors that the system must learn to ignore.

## 8.3 Filtering

Two filters trim the raw data. First, the tokenized concatenation of evidence and article must fit in 1024 tokens, budgeted as 600 for evidence and 400 for the article. When the evidence runs over, snippets are truncated in increasing order of their mention overlap with the article (Section 7.3), so the least related snippet loses text first. The frozen model accepts longer inputs, but shorter sequences make every forward pass cheaper, and intro-section articles rarely need more. An instance whose article alone exceeds the budget leaves the dataset, which removes the longest articles and biases the test toward short ones. The cap is there for speed, not for any limit of the model, so if the retention report shows heavy losses on gold, we raise the cap and accept slower passes. Second, the update must modify or remove at least one existing sentence. Updates that only *append* new sentences are excluded, because inserting a sentence means choosing where to open an all-m block, a placement decision our current design does not make. This is a scope cut, and we will report what fraction of the data each filter removes.

We hold out two thousand silver articles as the validation set. The remaining silver articles train the heads, and gold is the test set. Thresholds that concern one head alone ($\tau$, $\tau_c$, $\tau_d$) are tuned against derived labels on the full validation set. Settings whose effect only shows through the whole loop ($\lambda$, $\tau_2$, the deletion margin, the round cap) are tuned on a fixed slice of 200 validation articles, with UpdateROUGE against the silver revision as the tuning objective. Silver labels are noisy, so a threshold tuned against them can be systematically off. As a check, we will re-tune $\tau$ on the one hundred audited articles of Section 8.4 and report both values.

## 8.4 Label Derivation

The raw data gives article pairs $(A, A^*)$, but the heads need per-sentence and per-token labels. We derive them by alignment. Sentence boundaries come from the spaCy sentence splitter, the same library as the mention extractor, and every per-sentence decision in the system inherits its mistakes, so the audit below also records obvious mis-splits.

*Sentence alignment.* For every source sentence $s$ and target sentence $s'$, compute a similarity score, the F1 overlap of their token multisets, which is the harmonic mean of the fraction of $s$'s tokens present in $s'$ and the fraction of $s'$'s tokens present in $s$. Match pairs greedily from highest similarity down, each sentence used at most once. After matching, a source sentence with an exact match, or similarity at least 0.95, is KEEP; a source sentence matched with similarity at least 0.35 is EDIT; an unmatched source sentence is DROP. Unmatched *target* sentences are additions, excluded from labels by the scope cut of Section 8.3.

*Token labels.* Inside every EDIT pair, a token-level diff (the longest-common-subsequence algorithm familiar from version control) marks which source tokens the revision replaced or removed. These tokens are stale, and the rest are not. Insertions at a span boundary attach to the nearest stale token, so span expansion at inference time can express them.

Alignment is a heuristic and it will make mistakes, most visibly on sentences rewritten so heavily that they resemble a deletion plus an unrelated addition. Before any training run consumes these labels, we will audit one hundred aligned articles by hand and record how often we disagree with the automatic labels. If the audit shows the aligner frequently confusing heavy rewrites with DROP-plus-addition, we will switch the similarity to a character-level F-score, which is more tolerant of rewordings that reuse word stems, and re-derive all labels.

### 8.5 Cascade Pair Mining

Consequence prediction (Section 6.4) needs training pairs, and the cascade evaluation (Section 9.2) needs a labeled test set. No dataset provides either. The two are built by different procedures, because building both with one mining rule would grade every detector on a population selected by that rule's own criterion.

Both procedures start from the same definitions. Let $D_j$ be the *changed mentions* of sentence $j$, the strings that appear in the mention set of exactly one of the sentence's two versions, using the extractor $M$ of Section 7.3. An edited sentence is *direct* when $D_j$ shares a string with the mentions of the evidence, meaning its changed content appears there. Every sentence that changed but is not direct is either a cascade or a stylistic rewrite.

*Evaluation set.* From the gold articles, we sample several hundred changed-but-not-direct sentences and hand-label each one as cascade or stylistic. The labeled cascades split into two groups, those that share a mention string with a direct edit of the same article and those that do not. Cascade recall (Section 9.2) is reported on each group separately. Detector (a) scores zero on the no-shared-mention group by construction, and that group is where consequence prediction is measured. The direct-or-not split is itself automatic, and a genuine cascade whose changed mentions happen to match an evidence string is classified direct and never reaches the hand-labeling. We therefore also hand-check one hundred automatically-direct sentences and report how many are mislabeled cascades.

*Training pairs.* Requiring a shared mention string would exclude every paraphrase cascade from training, so positives are mined by meaning instead. For each changed-but-not-direct silver sentence, compare the text its edit changed with the text each direct edit changed, using an off-the-shelf sentence encoder, a model that maps text to vectors so that vector closeness tracks closeness of meaning. Pairs above a validation-tuned similarity threshold become positives. Before anything trains on the mined pairs, we hand-label two hundred of them. If fewer than roughly sixty percent are genuine cascades, the pairs are too noisy to train on, so consequence prediction leaves the plan and detectors (a) and (b) proceed. String-overlap mining, which admits only shared-mention pairs, stays in the ablation grid as the alternative scheme. No evaluation sentence enters training.

## 9 Evaluation

A single score cannot tell us what we need to know, because the system can fail in independent ways: it can miss stale text, repair it wrongly, rewrite text that was correct, or ignore cascades. This section defines one measurement per failure mode, after a short primer on the standard quantities.

## 9.1 Precision and Recall

Suppose the token detector marks some tokens stale, and compare its marks with the true labels. *Precision* is the fraction of marked tokens that are truly stale. *Recall* is the fraction of truly stale tokens that the detector marked. The two trade off through the threshold $\tau$. Masking more aggressively raises recall and lowers precision. The *F1 score* is their harmonic mean, a single number that is high only when both are. For a three-way classifier, *macro-F1* computes F1 for each class separately and averages the three, so the frequent KEEP class does not dominate the average.

Any fixed threshold shows one point of the trade-off. Sweeping the threshold and plotting precision against recall gives the *precision-recall curve*. The area under it summarizes detector quality across all thresholds and lets us compare detectors without committing to a single $\tau$.

## 9.2 Metrics

Let $U(\cdot)$ select an article's *updated* sentences, the ones with no exact match in the original $A$ after whitespace normalization. The gold updated sentences are $U(A^*)$ and the system's are $U(\hat{A})$.

- *UpdateROUGE* (Logan IV et al., 2022). ROUGE counts overlapping word sequences between a system text and a reference text. ROUGE-1 counts shared single words, ROUGE-2 shared two-word sequences, and ROUGE-L the longest shared subsequence. Each is reported in its F1 form, combining the overlap measured in both directions. Whole-article ROUGE is useless here, because the untouched bulk of the article overlaps perfectly and hides the part that matters, so UpdateROUGE computes all three scores between $U(\hat{A})$ and $U(A^*)$ only. We treat UpdateROUGE as the primary quality metric.
- *Entity precision and recall.* Extract mentions from the updated sentences with the mention extractor $M$ of Section 7.3, on both our output and the gold article. Precision is the fraction of our mentions that also appear among the gold mentions. Recall is the fraction of gold mentions that appear among ours. Updates mostly change names, numbers, and dates, and these two numbers target exactly those tokens.
- *Unsupported entity tokens.* The count of entity tokens in $U(\hat{A})$ that appear in neither the original article nor the evidence. If the system writes that the fire happened in *March* and no input text contains *March*, that token counts. Every such token is a fabrication, so lower is better and zero is the goal.
- *Faithfulness.* AlignScore (Zha et al., 2023) between the updated sentences $U(\hat{A})$ and the evidence. AlignScore splits the evidence into chunks, scores each sentence against the chunks with a model trained on a broad mix of entailment and question-answering data, and aggregates the results into one number between zero and one. The metric is restricted to updated sentences for the same reason UpdateROUGE is. Kept sentences discuss content the evidence never mentions and would score near zero no matter what the system did, which would dilute the number.
- *Minimality.* The token-level *Levenshtein distance* between $\hat{A}$ and $A$, the minimum count of single-token insertions, deletions, and substitutions that turns one into the other, normalized by the longer length. Alongside it we report the fraction of gold-KEEP sentences the output reproduces verbatim. Minimality is read together with the update metrics, never alone, since a system that edits nothing scores a perfect zero distance.
- *Detection quality.* For the sentence classifier, macro-F1 over the three classes against labels derived from the gold revision pairs. For the token scorer, precision, recall, and the area under the precision-recall curve against the derived token labels.
- *Cascade recall.* On the hand-labeled cascade set of Section 8.5, the fraction of cascade sentences that the active detector flags in any round, reported separately for cascades that share a mention with a direct edit and for cascades that do not, plus UpdateROUGE restricted to cascade sentences. This is the direct test of the iterative loop.

Every metric above is automatic, and the reference-based ones compare against a single gold text, so a correct update phrased differently from the gold reference is penalized. To measure that effect, we will read fifty

system outputs next to their gold articles, tally the errors by type (wrong fact, missed update, unnecessary edit, broken fluency), and report the tally next to the automatic numbers, following the error analysis format of Logan IV et al. (2022).

## 9.3 Baselines and Oracles

Every system below sees the same filtered gold instances and is scored with every metric of Section 9.2.

*Copy.* Return $A$ unchanged. Copy scores perfectly on minimality, reproduces every gold-KEEP sentence, and makes no updates.

*Full regeneration.* The input sequence is the evidence, then the outdated article, then a block of m tokens. The frozen sampler writes a complete updated article into the masked block, reading both the evidence and the old article as context. Nothing is preserved and everything is generated, the opposite of the editing pipeline. Because the block length fixes the output length, we generate at three target lengths, the original article length and 20% shorter and longer, and keep the best by PLL (Equation (11)). Regeneration is the opposite extreme from copy, scoring on the update metrics and not on minimality.

*Fine-tuned editor.* A T5 model fine-tuned on our filtered silver training split in the style of EDIT5 (Logan IV et al., 2022), reading the evidence and the outdated article and emitting the updated article, with copy tokens standing in for unchanged sentences. This is the published approach on this data. Its backbone predates our gold window while the diffusion model postdates it, a mismatch Section 9.6 returns to.

*Prompted model.* An instruction-tuned autoregressive model of size comparable to ours, given a fixed prompt containing the evidence and the article and asked for the corrected article, decoding greedily. This is the point of comparison for solving the task with prompting alone.

*No-evidence control.* The full DiffusEdit pipeline with the evidence deleted from its input. Whatever it still fixes comes from the frozen model's own memory rather than from the evidence; Section 9.6 explains why this control matters.

*Oracles.* An oracle run replaces one stage's predictions with the true answers from the derived labels, while every other stage runs as normal. We run four. With *oracle sentence labels*, the classifier is skipped and the derived KEEP/EDIT/DROP labels feed the masking stage directly. With *oracle token masks*, the token scorer is skipped and exactly the derived stale tokens are masked inside the true EDIT sentences, with the loop continuing from infilling. With *oracle cascade flags*, the detector is skipped and each round flags exactly the hand-labeled cascade sentences from Section 8.5. With *oracle reranking*, the masks and candidates stay learned and the candidate closest to the gold text wins. The gap between oracle reranking and the learned reranker separates a candidate pool that lacks the right answer from a reranker that fails to pick it. An oracle is an upper bound, and the gap between an oracle run and the corresponding learned run measures how many points that stage's errors cost end to end, which tells us where to spend future effort.

## 9.4 Ablations

An *ablation* removes or varies one component while holding the rest fixed, to measure that component's contribution. Table 2 lists the grid, and each row varies one axis against the bolded defaults. With the cascade detector off, the system collapses to a single-pass, article-scale Detect, Remask, Repair, so this row measures what iteration adds. Ablation rows run on a fixed random subsample of 200 gold articles. The default configuration, the baselines, and the controls run on the full gold set.

## 9.5 Success Criteria

The experiments test three claims. With a cascade detector on, cascade recall and UpdateROUGE on cascade sentences must exceed the single-pass numbers. DiffusEdit must reach the fine-tuned editor's UpdateROUGE, within the reported uncertainty, at no more than half the editor's normalized edit distance. The factor of one half is a bar chosen before the experiments, not a derived quantity. The full system must beat its own no-evidence control on the update metrics. Each claim can fail, and when time runs short, the experiments run in this order.

**Table 2** Ablation axes. Defaults in bold. Each experiment varies one axis.

| Axis | Values |
|---|---|
| Cascade detector | none (single pass), (a), **(b)**, (c) |
| Rounds $R$ | 1, 2, **3**, 4 |
| Candidate lengths $\Lambda$ | $\{\ell\}$, $\{0, \ell\}$, **full** |
| Detector training data | synthetic, derived, **mix** |
| Reranker | PLL only, verifier only, **combined** |
| Deletion margin | off, **on** |
| Cascade mining | string overlap, **embedding** |

### 9.6 Validity

Two problems could distort the reported numbers.

*Chance.* With 914 test articles, a small gap between two systems can come from luck rather than quality. We do not claim a win from a raw difference alone. Every reported comparison will carry an uncertainty estimate from a standard significance test over per-article scores, and gaps the test cannot separate from chance will be reported as ties. All thresholds ($\tau$, $\tau_2$, $\tau_c$, $\tau_d$, $\lambda$, and the deletion margin) are chosen on validation data through the procedure of Section 8.3 and frozen before anything touches the test set.

*Pretraining contamination.* The frozen model's training data almost surely includes Wikipedia, possibly including the 2020–2021 revisions the gold set is built from. If so, the model may write the correct update from memory, with or without our evidence, and the scores would then reflect memory rather than use of the evidence. The no-evidence control of Section 9.3 measures this directly, because anything it fixes without seeing evidence must come from memory. If the control scores high, we will report it and treat the gap between the full system and the control, the *evidence-conditioned gain*, as the measure of what the system adds.

There is a related asymmetry between the baselines. The fine-tuned editor's backbone predates the 2020–2021 gold window, while the diffusion model and the prompted model postdate it, so the newer models may hold some answers in memory that the older one cannot. Pinning the diffusion model to a checkpoint from the editor's era would remove the asymmetry, but no such checkpoint exists. In mid-2022, discrete diffusion language models were research prototypes of roughly BERT scale, around one hundred million parameters (Austin et al., 2021; He et al., 2022). Masked diffusion models at the billion-parameter scale arrived with LLaDA in 2025 (Nie et al., 2025). The control above is the substitute. We run the no-evidence control for every model-backed system, and across model generations we compare evidence-conditioned gains rather than raw scores.

## 10 Risks and Limitations

1. *Evidence may not steer the generator.* The frozen model was never trained to treat a prepended snippet as overriding the fluent text around a masked span, so infilling may regenerate the stale content and give the reranker a candidate pool with no faithful member, which reranking cannot repair. Detect, Remask, Repair reports frozen-model repair working on short summaries, and whether that transfers to article length is untested. The first implementation milestone measures it directly. Mask exactly the derived stale tokens on gold pairs, infill with the frozen model, and record the fraction of spans where any candidate contains the gold fact, with and without the evidence in the input. If the with-evidence rate is low, the escalation options, in order, are the instruction-tuned checkpoint with the evidence formatted as an instruction, a decoding-time score bonus for tokens that appear in the evidence, reward steering (Section 7.2), and a small fine-tuned adapter for the repair step. The adapter departs from the frozen-model design and would be reported as such.
2. *Silver label noise.* About two thirds of silver edits lack support in their attached evidence, so the derived EDIT labels overcount. Three defenses are already in the design: the synthetic corruption stream, the

hundred-article label audit, and gold-only reporting. If the audit comes back worse than expected, we lean the detector training mix toward synthetic data.

3. *Detector shortcut.* The synthetic stream labels model-refilled tokens stale, so the token scorer could learn to recognize model-generated text rather than evidence contradiction. The training-data ablation measures this. A scorer trained only on synthetic data that scores well on synthetic validation and poorly on derived labels has learned the shortcut, and the training mix then leans toward derived data.

4. *Alignment brittleness.* If the label audit shows the aligner frequently mistaking a heavy rewrite for a deletion plus an addition, we switch the similarity from token-multiset F1 to a character-level F-score and re-derive all labels before training anything on them.

5. *Cascade mining noise.* If the hand-check of mined pairs lands below the bar of Section 8.5, consequence prediction leaves the experiment plan and the loop rests on detectors (a) and (b).

6. *Insertion scope.* Excluding pure additions narrows the task, and the filter retention numbers will show by how much. The cut also limits cascade repair, since a cascade whose correct fix is a new sentence cannot be repaired, and a DROP decision is not revisited in later rounds. The natural extension is a placement head that chooses where to open an all-m block. It can reuse the pooled sentence vectors, and we keep it out of scope until the edit loop works.

7. *Consequence input.* Should consequence prediction read the changed sentence *before* or *after* its rewrite? Both vectors exist at round end. We start with after, because the rewritten content is what the dependent sentence must now agree with, and revisit the choice if cascade recall is low.

## References


Jacob Austin, Daniel D. Johnson, Jonathan Ho, Daniel Tarlow, and Rianne van den Berg. Structured denoising diffusion models in discrete state-spaces. In *Advances in Neural Information Processing Systems*, volume 34, 2021. URL https://arxiv.org/abs/2107.03006.

Tiwei Bie et al. Llada2.1: Speeding up text diffusion via token editing, 2026. URL https://arxiv.org/abs/2602.08676.

Jiangjie Chen et al. Converge to the truth: Factual error correction via iterative constrained editing, 2022. URL https://arxiv.org/abs/2211.12130.

Luyu Gao, Zhuyun Dai, Panupong Pasupat, Anthony Chen, Arun Tejasvi Chaganty, Yicheng Fan, Vincent Zhao, Ni Lao, Hongrae Lee, Da-Cheng Juan, and Kelvin Guu. RARR: Researching and revising what language models say, using language models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 16477–16508, Toronto, Canada, July 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.acl-long.910. URL https://aclanthology.org/2023.acl-long.910/.

Zhengfu He, Tianxiang Sun, Kuanning Wang, Xuanjing Huang, and Xipeng Qiu. Diffusionbert: Improving generative masked language models with diffusion models, 2022. URL https://arxiv.org/abs/2211.15029.

Hayate Iso, Chao Qiao, and Hang Li. Fact-based text editing. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 171–182, Online, July 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.acl-main.17. URL https://aclanthology.org/2020.acl-main.17/.

Kundan Krishna, Sanjana Ramprasad, Prakhar Gupta, Byron C. Wallace, Zachary C. Lipton, and Jeffrey P. Bigham. Genaudit: Fixing factual errors in language model outputs with evidence, 2024. URL https://arxiv.org/abs/2402.12566.

Garvin Kruthof. Editpropbench: Measuring factual edit propagation in scientific manuscripts, 2026. URL https://arxiv.org/abs/2605.02083.

Robert L. Logan IV, Alexandre Passos, Sameer Singh, and Ming-Wei Chang. FRUIT: Faithfully reflecting updated information in text. In Marine Carpuat, Marie-Catherine de Marneffe, and Ivan Vladimir Meza Ruiz, editors, *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 3670–3686, Seattle, United States, July 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.naacl-main.269. URL https://aclanthology.org/2022.naacl-main.269/.

Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in gpt. In *Advances in Neural Information Processing Systems*, volume 35, 2022. URL https://arxiv.org/abs/2202.05262.

Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, Jun Zhou, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. Large language diffusion models, 2025. URL https://arxiv.org/abs/2502.09992.

Timo Schick, Jane Dwivedi-Yu, Zhengbao Jiang, Fabio Petroni, Patrick Lewis, Gautier Izacard, Qingfei You, Christoforos Nalmpantis, Edouard Grave, and Sebastian Riedel. Peer: A collaborative language model, 2022. URL https://arxiv.org/abs/2208.11663.

Tal Schuster, Adam Fisch, and Regina Barzilay. Get your vitamin C! robust fact verification with contrastive evidence. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 624–643, Online, June 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.naacl-main.52. URL https://aclanthology.org/2021.naacl-main.52/.

Darsh J. Shah, Tal Schuster, and Regina Barzilay. Automatic fact-guided sentence modification. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 8791–8798, 2020. doi: 10.1609/aaai.v34i05.6406. URL https://doi.org/10.1609/aaai.v34i05.6406.

Yu Shi et al. Hicam: A hierarchical-causal modification framework for long-form text modification, 2025. URL https://arxiv.org/abs/2505.24319.

James Thorne and Andreas Vlachos. Evidence-based factual error correction. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 3298–3309, Online, August 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.acl-long.256. URL https://aclanthology.org/2021.acl-long.256/.

Guanghan Wang, Yair Schiff, Subham Sekhar Sahoo, and Volodymyr Kuleshov. Remasking discrete diffusion models with inference-time scaling, 2025. URL https://arxiv.org/abs/2503.00307.

Zirui Wu et al. Dreamon: Diffusion language models for code infilling beyond fixed-size canvas, 2026. URL https://arxiv.org/abs/2602.01326.

Lin Yao. Remask, don't replace: Token-to-mask refinement in diffusion large language models, 2026. URL https://arxiv.org/abs/2604.18738.

Jiacheng Ye, Zhihui Xie, Lin Zheng, Jiahui Gao, Zirui Wu, Xin Jiang, Zhenguo Li, and Lingpeng Kong. Dream 7b: Diffusion large language models, 2025. URL https://arxiv.org/abs/2508.15487.

Yuheng Zha, Yichi Yang, Ruichen Li, and Zhiting Hu. AlignScore: Evaluating factual consistency with a unified alignment function. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 11328–11348, Toronto, Canada, July 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.acl-long.634. URL https://aclanthology.org/2023.acl-long.634/.

Andrew Zhang et al. Flexible-length text infilling for discrete diffusion models, 2025. URL https://arxiv.org/abs/2506.13579.

Hao Zou, Zachary Horvitz, Chandhru Karthick, Zhou Yu, and Kathleen McKeown. Detect, remask, repair: Diffusion editing for faithful summarization of evolving contexts, 2026. URL https://arxiv.org/abs/2606.12807.